

Fundamentals of Programming

CS-114

Week - 12

Strings & File Handling

A Word About Character Arrays

char arr[100];

⇒ cin >> arr,
↑

- Character arrays enable us to work with words/phrases
 - Input terminates as soon as a whitespace character is entered

cin.get(arr, 100);

- Different syntax needed to make it work with whitespaces

→ `cin.getline(array_name, array_length)`

Or

→ `cin.get (array_name, array_length)`

Strings

```
char c = 'c';  
char arr[100] = "This is a dog";  
⇒ string firstName = "Muhammad Ahmed"; cin >> ; // whitespace terminate  
cin.get(arr, 100);
```

- What if we want to add other functionality on top of character arrays?
- We can use the string library in C++ (two options available):
 - #include<cstring> (C-style strings. Offers a number of handy functions)
 - #include<string> (Simplifies the usage of strings even further)

- String libraries allow us to manipulate strings in a handful of ways
 - Compare strings 
 - Search strings for characters and other strings
 - Tokenize strings (separate strings into logical pieces)

- cstring built-in functions (methods) also work with character arrays

The C++ `string` Class

- Special datatype supports working with strings
- `#include <string>` 
- Can define `string` variables in programs:

```
string firstName, lastName;
```

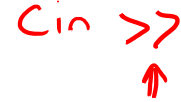


- Can receive values with assignment operator:

```
firstName = "George";  
lastName = "Washington";
```

- Can be displayed via `cout`

```
cout << firstName << " " << lastName;
```



string Operators

string s_1, s_2, s_3
 $s_1 = s_2 + s_3$



OPERATOR	MEANING
>>	extracts characters from stream up to whitespace, insert into string
<<	inserts string into stream
=	assigns string on right to string object on left
+=	appends string on right to end of contents on left
+	concatenates two strings ←
[]	references character in string using array notation
>, >=, <, <=, ==, !=	relational operators for string comparison. Return true or false

string Comparison

- Can use relational operators directly to compare string objects:

```
string str1 = "George",  
        str2 = "Georgia";  
→ if (str1 < str2)  
    cout << str1 << " is less than "  
        << str2;
```

- Comparison is performed similar to `strcmp` function.
Result is `true` or `false`

string Operators

array of character

```
string word1, phrase;
string word2 = "-Dog";
cin >> word1;    // lion
phrase = word1 + word2;
phrase += " on a bun";
for (int i = 0; i < 16; i++)
    cout << phrase[i]; // displays
```

lion Dog on a bu

string Member Functions



"med"

- Are behind many overloaded operators

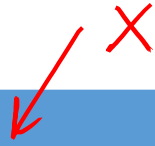
Almed

- Categories:

- assignment: assign, copy, data
- modification: append, clear, erase, insert, replace, swap
- space management: capacity, empty, length, resize, size
- substrings: find, substr
- comparison: compare



cstring vs string methods

Functionality	 cstring	 string
Copies the string s2 into the character array s1 . The value of s1 is returned.	<code>char *strcpy(char *s1, const char *s2);</code>	<code>s1 = s2</code>
Copies at most n characters of the string s2 into the character array s1 . The value of s1 is returned.	<code>char *strncpy(char *s1, const char *s2, size_t n);</code>	<code>s1 = s2.substr(start_pos, number_of_char)</code>
Appends the string s2 to the string s1 . The first character of s2 overwrites the terminating null character of s1 . The value of s1 is returned.	<code>char *strcat(char *s1, const char *s2);</code>	<code>s3 = s1+s2</code>
Appends at most n characters of string s2 to string s1 . The first character of s2 overwrites the terminating null character of s1 . The value of s1 is returned.	<code>char *strncat(char *s1, const char *s2, size_t n);</code>	<code>s2 = s1+ s2.substr(start_pos, number_of_char)</code>
Compares the string s1 with the string s2 . The function returns a value of zero, less than zero or greater than zero if s1 is equal to, less than or greater than s2 , respectively.	<code>int strcmp(const char *s1, const char *s2);</code>	<code>s1 == s2 s1 < s2 s1 >= s2</code>
Find out the length of a string	<code>int strlen(const char * s1);</code>	<code>s1.length()</code>

string Member Functions

```
string word1, word2, phrase;  
cin >> word1; //lion  
word2.assign(" Dog");  
phrase.append(word1);  
phrase.append(word2);  
cout << phrase << endl;  
phrase.append(" with mustard relish", 13);  
cout << phrase << endl;  
phrase.insert(8, "on a bun ");  
cout << phrase << endl;
```

```
lion Dog  
lion Dog with mustard  
lion Dogon a bun  with mustard
```

Example

```
1
2 // Using strcpy and strncpy.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 #include <cstring> // prototypes for strcpy and strncpy
9
10 int main()
11 {
12     char x[] = "Happy Birthday to You";
13     char y[ 25 ];
14     char z[ 15 ];
15
16     strcpy( y, x ); // copy contents of x into y
17
18     cout << "The string in array x is: " << x
19           << "\nThe string in array y is: " << y << '\n';
20
21     // copy first 14 characters of x into z
22     strncpy( z, x, 14 ); // does not copy null character
23     z[ 14 ] = '\0';      // append '\0' to z's contents
24
25     cout << "The string in array z is: " << z << endl;
26
27     return 0; // indicates successful termination
28
29 } // end main
```

The string in array x is: Happy Birthday to You
The string in array y is: Happy Birthday to You
The string in array z is: Happy Birthday

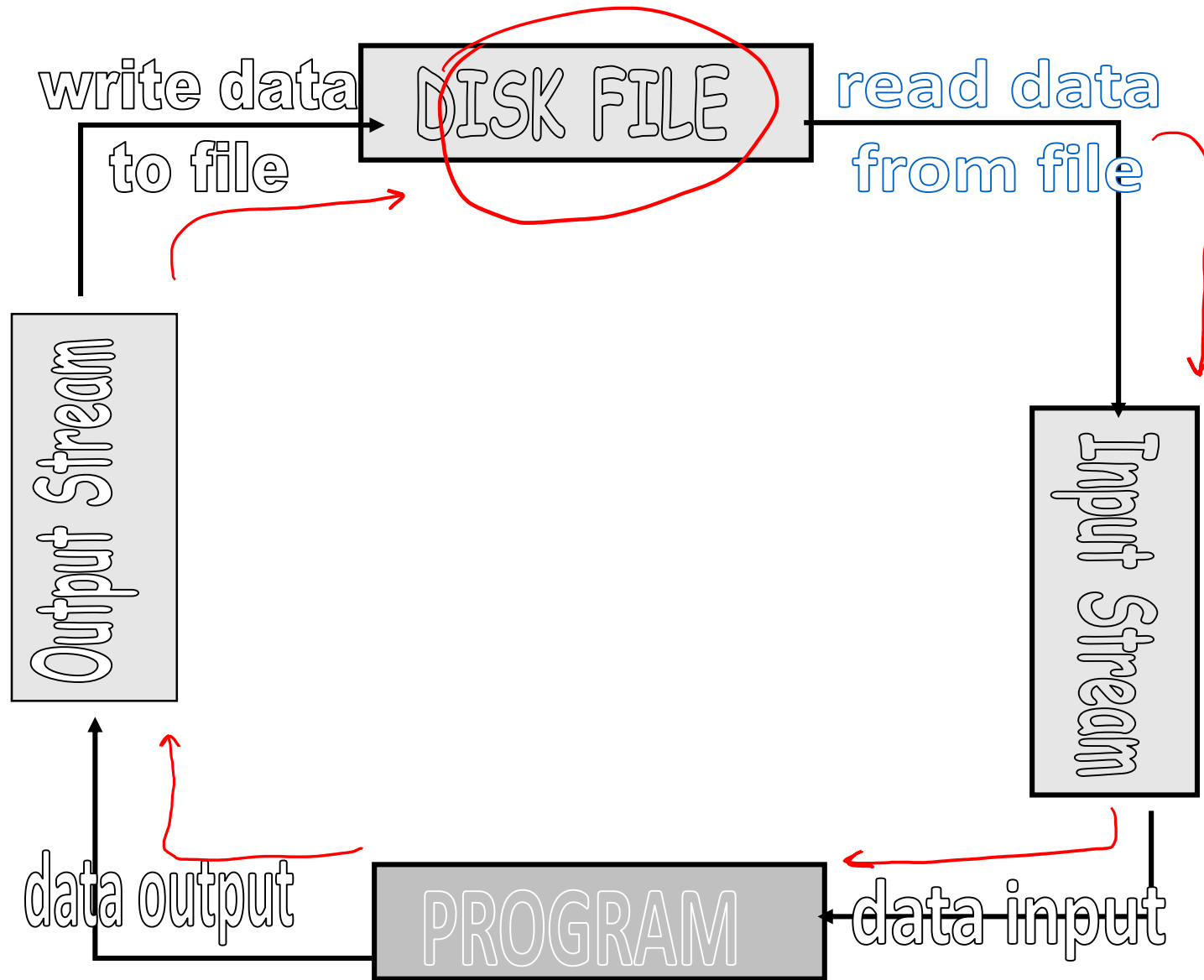
File Handling

File Handling

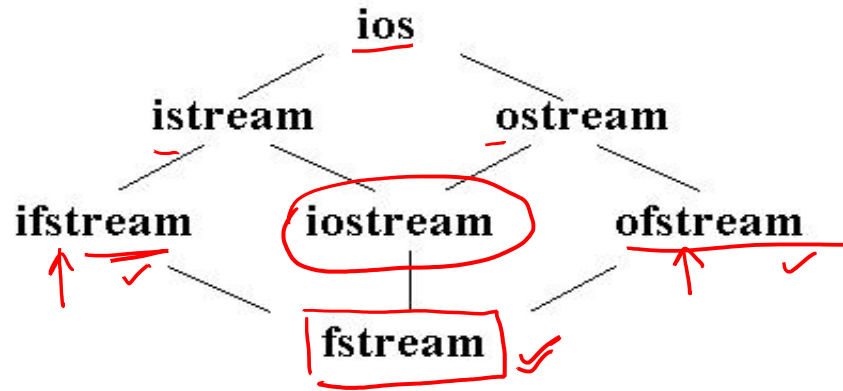
- So far:
 - In all programs, we have taken input from keyboard
 - Manipulated it in some way and displayed it on the monitor
- As soon as the programme terminates, all our variables/data lost
- Can be difficult to enter large amount of input data
- The solution: use File Handling

Using Input/Output Files

- Files in C++ are interpreted as a sequence of bytes stored on some storage media.
- The data of a file is stored in either readable form or in binary code called as text file or binary file.
- The flow of data from any source to a sink is called as a stream
- Computer programs are associated to work with files as it helps in storing data & information permanently.
- File - itself a bunch of bytes stored on some storage devices.
- In C++ this is achieved through a component header file called `fstream.h`
- The I/O library manages two aspects- as interface and for transfer of data.
- The library predefine a set of operations for all file related handling through certain classes.



Classes for Stream I/O in C++



ios is the base class.
istream and ostream inherit from ios
ifstream inherits from istream (and ios)
ofstream inherits from ostream (and ios)
iostream inherits from istream and ostream (& ios)
fstream inherits from ifstream, iostream, and ofstream

When working with files in C++, the following classes can be used:

ofstream - writing to a file

ifstream - reading for a file

fstream - reading / writing

When ever we include <iostream.h>, an ostream object, is automatically defined - this object is cout.

ofstream inherits from the class ostream (standard output class).

ostream overloaded the operator >> for standard output....thus an ofstream object can use methods and operators defined in ostream.

```
#include <fstream.h>
```

```
int main (void)
```

```
{
```

```
// Local Declarations
```

```
→ ifstream    fsIn;
```

```
    ofstream    fsOut;
```

```
•
```

```
•
```

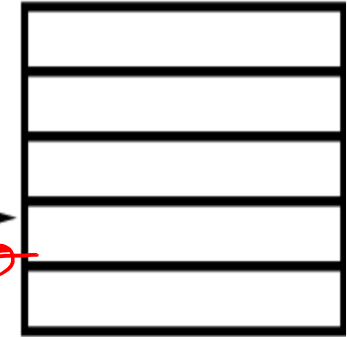
```
•
```

```
} // main
```

fsIn is an input
instance of ifstream



fsIn

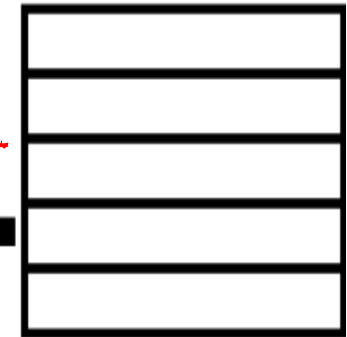


memory

fsOut is an output
instance of ofstream



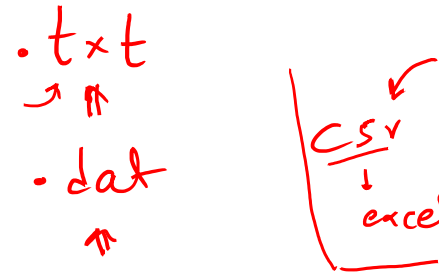
fsOut



memory

Using Input/Output Files

- **stream** - a sequence of characters
 - interactive (iostream)
 - **cin** - input stream associated with **keyboard**.
 - **cout** - output stream associated with **display**.
 - file (fstream)
 - **ifstream** - defines new input stream (normally associated with a file).
 - **ofstream** - defines new output stream (normally associated with a file).



- Stream of bytes to do input and output to different devices.
- Stream is the basic concepts which can be attached to files, strings, console and other devices.
- User can also create their own stream to cater specific device or user defined class.

File Handling

- File handling allows us to permanently store the data in text or binary files
- We use certain libraries to write/read data to a file
 - `#include <fstream>` (This is what you will be using)
 - `#include <ofstream>`
 - `#include <ifstream>`

General File I/O Steps

- 
- Declare a file name variable ←
 - Associate the file name variable with the disk file name
 - Open the file ←
 - Use the file ←
 - Close the file ←

```

#include <fstream>
#include <iostream>
using namespace std;
int main ()
{
char data[100];
// open a file in write mode.
ofstream outfile;
outfile.open("afile.dat");
cout << "Writing to the file" << endl;
cout << "Enter your name: ";
cin.getline(data, 100);
// write inputted data into the file.
outfile << data ;
cout << "Enter your age: ";
cin >> data;
// again write inputted data into the file.
outfile << data ;
// close the opened file.
outfile.close();

```

```

// open a file in read mode.
ifstream infile;
infile.open("afile.dat");
cout << "Reading from the file" << endl;
while(1) //reading complete data from file
{
infile >> data;
// write the data at the screen.
cout << data << endl;
if(infile.eof())
break;
}
infile.close();
return 0;
}

```

Class 67%
 - Quiz: $\frac{100\%}{20\%}$
 Assignment:
 Final: 50%
 CP: 30%

Lab 33%
 Labs: 50%
 O.L: 10%
 L.F: 10%
 Proj: 30
 ↓
10 Marks